Zadanie 1. Mamy do dyspozycji procedurę BIASED-RANDOM, która zwraca wartość 0 z prawdopodobieństwem p, zaś wartość 1 z prawdopodobieństwem $1 - p$, gdzie p jest pewną liczbą z przedziału $(0, 1)$. Zaproponuj algorytm, który, wykorzystując BIASED-RANDOM jako procedurę pomocniczą, zwraca 0 z prawdopodobieństwem $p/(1 + p)$ oraz 1 z prawdopodobieństwem $1/(1 + p)$. Jaki jest oczekiwany czas działania tego algorytmu w zależności od p?

Zadanie 2. Rozważmy problem sekretarki i załóżmy, że kandydaci przybywają w losowej kolejności. Jakie jest prawdopodobieństwo, że w procedurze „zatrudnij" zatrudnimy dokładnie dwie osoby?

Zadanie 3. Niech $A[1 : n]$ będzie losową permutacją zbioru $\{1, \ldots, n\}$. Dla ustalonego $k \geqslant 1$ parę (i, j), $i < j$, nazywamy *silną inwersją*, gdy $A[i] > A[j] + k$. Zastosuj zmienne losowe wskaźnikowe do obliczenia oczekiwanej liczby silnych inwersji.

Zadanie 4. Wykonujemy n-krotnie procedurę RANDOM$(1, n^3)$. Wyznacz wartość oczekiwaną liczby różnych otrzymanych wyników jako funkcję n. Następnie wyprowadź rozwinięcie asymptotyczne tej wartości z dokładnością do składnika rzędu $\mathcal{O}(1/n^2)$.

Zadanie 5. Przypuśćmy, że kule są wrzucane do b urn. Rzuty są niezależne i każda kula może trafić do każdej z urn z jednakowym prawdopodobieństwem. Jaka jest oczekiwana liczba rzutów, zanim co najmniej jedna z urn będzie zawierać dwie kule? Wystarczy dokładny wzór, ale spróbuj również wyznaczyć asymptotykę (powie nam znacznie więcej).

Zadanie 6. Ile osób powinno zostać zaproszonych na przyjęcie, aby oczekiwana liczba trójek osób urodzonych jednego dnia wyniosła co najmniej 1?

W każdym z zadań 7–9 scharakteryzuj zbiór permutacji rozmiaru n, które mogą zostać wygenerowane. Sprawdź ponadto, czy algorytm generuje możliwe do otrzymania permutacje zgodnie z rozkładem jednostajnym.

Zadanie 7.

PERMUTE-FIRST(A)

```
1: n ← A.length
2: for i ← 1 to n − 1 do
3:     swap(A[i], A[RANDOM(i + 1, n)])
4: end for
```

Zadanie 8.

PERMUTE-SECOND(A)

```
1: n ← A.length
2: for i ← 1 to n do
3:     swap(A[i], A[RANDOM(1, n)])
4: end for
```

Zadanie 9.

PERMUTE-THIRD(A)

```
1: n ← A.length
2: niech B[1 : n] będzie nową tablicą
3: offset ← RANDOM(1, n)
4: for i ← 1 to n do
5:     dest ← i + offset
6:     if dest > n then
7:         dest ← dest − n
8:     end if
9:     B[dest] ← A[i]
10: end for
11: return B
```